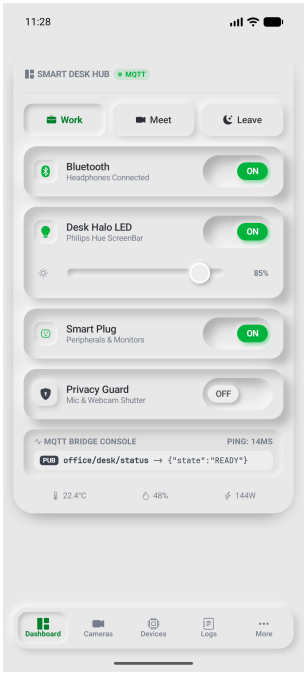
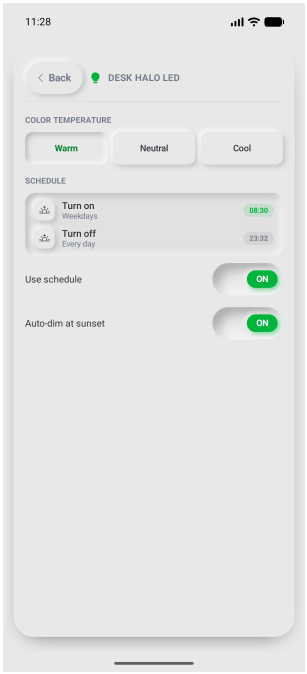
Smart Desk Hub

Kotlin ve Jetpack Compose ile native Android akıllı ofis uygulaması

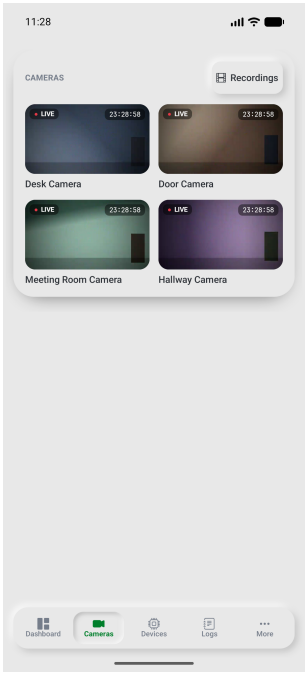
Masadaki cihazları açıp kapatan, sahneler çalıştıran, lambayı ve prizi otomatikleştiren, dört kamerayı izleyen, kayıtları oynatan, cihazları ve kullanıcıları yöneten ve her işlemi aranabilir bir log'da tutan bir akıllı ofis uygulaması. Proje, JavaFX + WebView ile yazılmış bir masaüstü widget'ı olarak başladı ve orijinalin neumorphic görünümü piksel piksel korunarak sıfırdan native Android uygulaması olarak yeniden yazıldı.



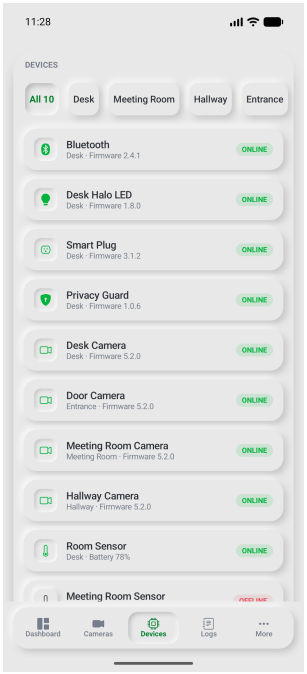
Dashboard



Lamba ayrıntıları



Kameralar



Cihazlar

Platform	Android 8.0+ (minSdk 26, targetSdk 37)
Dil ve arayüz	Kotlin 2.4 · Jetpack Compose · Material 3
Mimari	MVVM · tek Activity · Hilt ile bağımlılık enjeksiyonu
Veri ve arka plan	Room · DataStore · WorkManager · exact alarm
Medya ve widget	Media3 ExoPlayer · Jetpack Glance
Testler	323 birim ve Robolectric arayüz testi

Tüm cihazlar ve veriler simüle edilir; uygulama ağ bağlantısı olmadan çalışır. Kaynak kod: github.com/fmcoskun/smart-desk-hub · Lisans: MIT

1. Proje hakkında

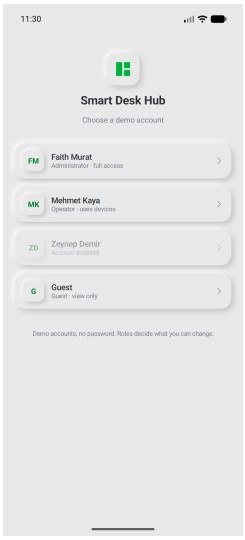
Smart Desk Hub, bir ofis masasındaki akıllı cihazları tek bir telefondan yönetmeyi gösteren bir portföy uygulamasıdır. Her şey telefonda, simüle edilmiş cihazlarla çalışır; bu yüzden herhangi bir ağ ya da sunucu gerektirmez.

Kod okunmak için yazıldı: her dosya ne olduğunu ve hangi katmana ait olduğunu söyleyen bir açıklamayla başlar, Android kavramları ilk kullanıldıkları yerde açıklanır.

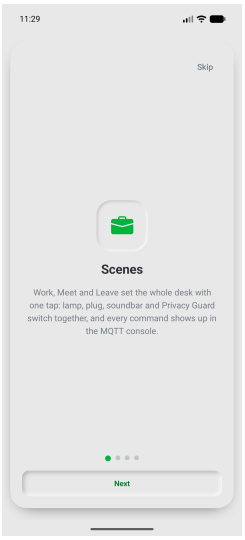
2. Özellikler

- **Dashboard** — Bluetooth soundbar, ışık ayarlı Desk Halo LED, Smart Plug ve Privacy Guard; **Work / Meet / Leave** sahneleri; canlı bir MQTT konsolu; sıcaklık, nem ve güç tüketimi.
- **Gerçekten çalışan ayrıntı sayfaları** — ses çıkışı, renk sıcaklığı, **prizin otomatik kapanma zamanlayıcısı**, **LED zamanlaması** ve **gün batımında kısma**. Bunlar uygulama kapalıyken de çalışır (AlarmManager exact alarm, yedek olarak WorkManager) ve çalıştıklarında bildirim gösterir.
- **Kameralar** — dört döngülü "canlı" görüntü; Privacy Guard açıkken masa kamerası kararır; anlık görüntü alma log'a yazılır.
- **Kayıtlar** — güne ve kameraya göre listeleme, PERSON/MOTION etiketleri; ileri-geri sarma, ±10 sn ve 1×/2× hız destekli Media3 oynatıcı; ekran döndürülünce kaldığı yerden devam eder.
- **Cihaz yöneticisi** — durum, firmware, IP/MAC, pil ve sinyal; yeniden adlandırma, oda değiştirme, yeniden başlatma (cihaz her yerde 5 saniye çevrimdışı görünür), ekleme ve silme.
- **Loglar** — her kullanıcı işlemi ve cihaz olayı canlı olarak; metin araması, kaynak / seviye / gün / cihaz filtreleri; paylaşım menüsüyle CSV olarak dışa aktarma.
- **Kullanıcılar ve roller** — Admin, Operator ve Guest, tek bir yerde tanımlanır; "son yönetici silinemez" ve "kendini silemezsin" kurallarıyla kullanıcı yönetimi.
- **Karşılama turu ve Kullanım Kılavuzu** — dört sayfalık karşılama turu ve aranabilir on yardım konusu.
- **Ana ekran widget'ı** (Jetpack Glance) — sahneler ve dört anahtar doğrudan ana ekranda, iki yönlü canlı; Dashboard ile aynı kurallar geçerli (Guest ve oturum açmamış kullanıcılar salt okunur görür, çevrimdışı cihazlar değiştirilmez).

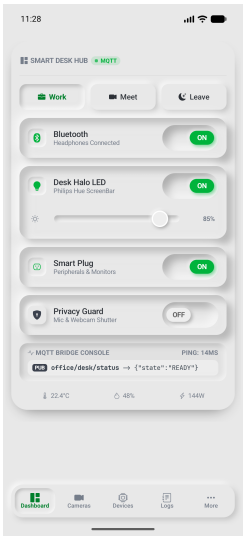
3. Ekran görüntüleri



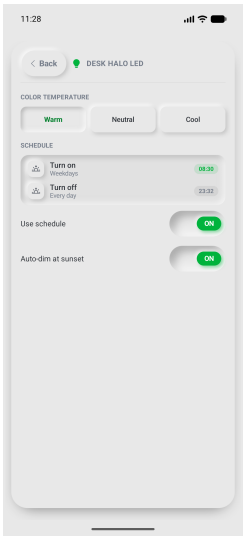
Giriş



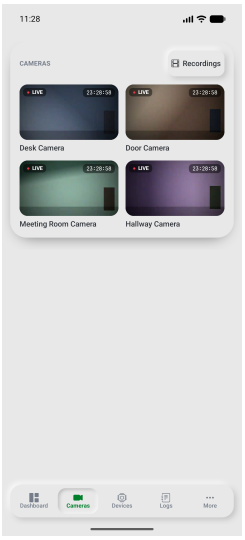
Karşılama turu



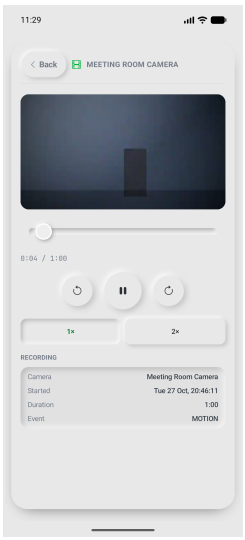
Dashboard



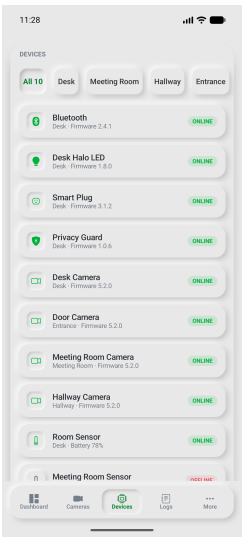
Lamba ayrıntıları



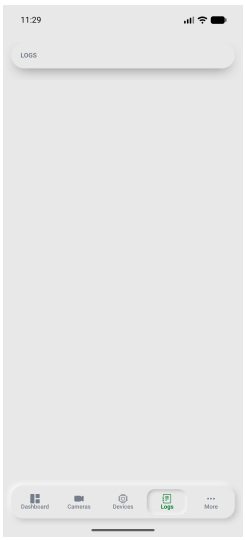
Kameralar



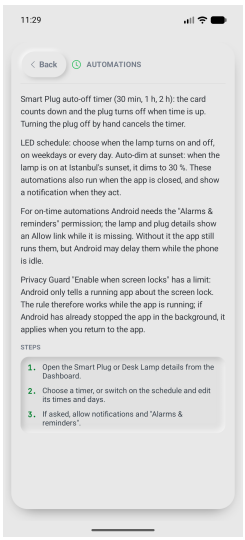
Kayıt oynatıcı



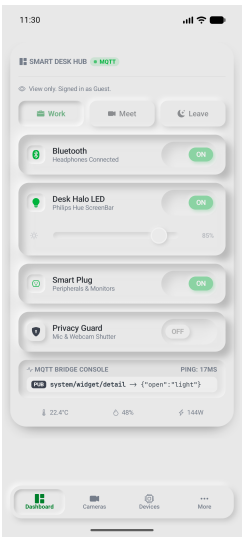
Cihazlar



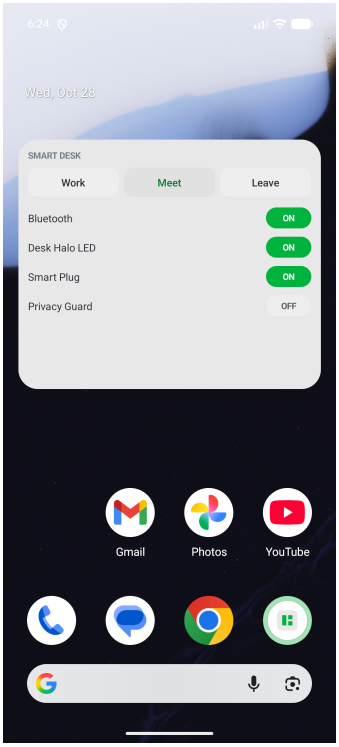
Loglar



Kullanım Kılavuzu



Guest görünümü



Ana ekran widget'ı

Sahne düğmeleri ve masanın dört anahtarı, uygulamayı açmadan ana ekrandan kullanılabilir. Widget'tan yapılan değişiklik uygulamada, uygulamadan yapılan değişiklik widget'ta anında görünür. Anahtarlama kuralları Dashboard ile ortak olan `DeskCommands` sınıfında tutulur.

4. Derleme ve çalıştırma

Gereksinimler: **JDK 17** ve **Android SDK** (Android Studio ikisini de kurar). Proje Gradle 9.8.2 (wrapper dahil), AGP 9.4.1 ve Kotlin 2.4.21 kullanır; `compileSdk`/`targetSdk` 37, `minSdk` 26.

```
./gradlew assembleDebug    # app/build/outputs/apk/debug/app-debug.apk dosyasını üretir
./gradlew test             # birim ve Robolectric arayüz testlerini çalıştırır (323 test)
```

Emülatörde: projeyi Android Studio'da açıp Run'a basın ya da bir emülatör başlatıp komut satırından kurun:

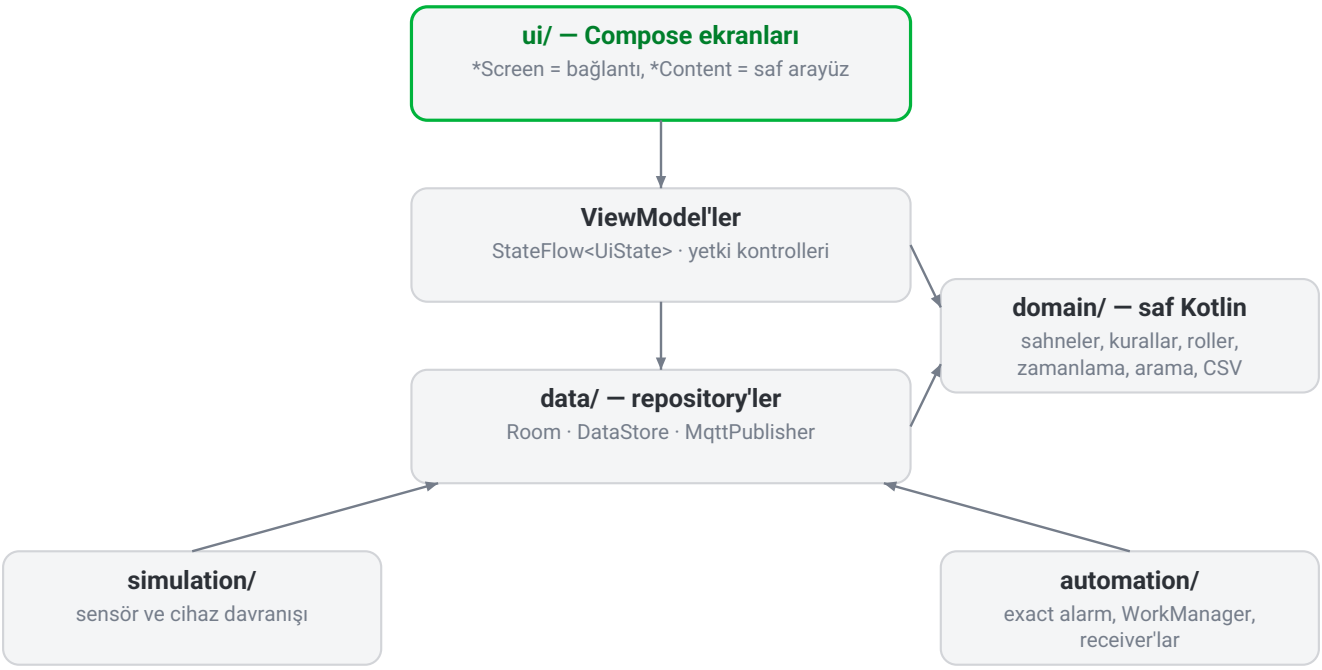
```
adb install -r app/build/outputs/apk/debug/app-debug.apk
adb shell am start -n com.iot.widget/.MainActivity
```

Telefonda: Ayarlar → Telefon hakkında → Yapım numarası'na yedi kez dokunarak Geliştirici seçeneklerini açın; ardından Ayarlar → Sistem → Geliştirici seçenekleri → USB hata ayıklama'yı etkinleştirin. Telefonu USB ile bağlayıp telefondaki onayı kabul edin, `adb devices` komutunun telefonu listelediğini kontrol edin ve aynı `adb install -r …` komutunu çalıştırın.

Giriş ekranında herhangi bir demo hesabı seçin (şifre yoktur). Admin hesabı her şeyi görür; Guest salt okunur deneyimi gösterir.

5. Mimari

Tek doğruluk kaynağı ilkesine dayanan MVVM: ekranlar ViewModel'lerdeki `StateFlow`'ları gösterir, ViewModel'ler repository'leri çağırır, verinin sahibi de repository'lerdir (Room ve DataStore). Hepsini Hilt birbirine bağlar.



Klasör haritası (`app/src/main/kotlin/com/iot/widget/`)

Klasör	İçinde ne var
`SmartDeskApp.kt`, `MainActivity.kt`	Uygulamanın başlangıcı (Hilt, WorkManager kurulumu, örnek veri) ve tek Activity
`di/`	Hilt modülü: her arayüze hangi gerçekleştirmenin verileceği
`domain/`	Saf Kotlin, Android yok: masa modelleri, sahneler, payload'lar, güç, biçimlendirme, Türkçe uyumlu isimler
`domain/users`, `domain/devices`	Rol → yetki tablosu, kullanıcı kuralları, cihaz kuralları, eşleştirme adımları
`domain/automation`	Zamanlama hesapları ve gün batımı hesaplayıcısı
`domain/logs`, `cameras`, `guide`	Log sorgusu / saklama / CSV, kayıt ve oynatma hesapları, kılavuz konuları ve arama
`data/`	Repository arayüzleri ve Room / DataStore gerçekleştirmeleri
`data/db`	Room entity'leri, DAO'lar, veritabanı (v5, otomatik migration, şemalar `app/schemas` altında)
`data/mqtt`, `data/log`	MQTT bağlantı noktası (ve her yayını log'a yazan bir decorator), olay log'u
`simulation/`	Oda sensörü ölçümleri, cihaz onayları, yeniden başlatmalar ve ara sıra uyarılar
`control/`	`DeskCommands`: Dashboard ve widget'ın ortak kullandığı anahtarlama kuralları
`widget/`	Glance ana ekran widget'ı, dokunma eylemleri ve Hilt giriş noktası
`automation/`	Zamanlama (exact alarm ya da WorkManager), worker'lar, açılış/izin receiver'ları, ekran kilidi, bildirimler
`ui/theme`, `components`, `icons`	Renkler, fontlar, neumorphic gölgeler, ortak kontroller, Bootstrap Icons fontu
`ui/<özellik>`	Her ekran için bir klasör: dashboard, detail, login, onboarding, users, devices, cameras, recordings, logs, guide, more
`ui/navigation`, `ui/shell`	Tip güvenli rotalar ve navigasyon grafiği; kök (giriş / tur / uygulama) ve alt menü

Bir dokunuşun yolculuğu

Dashboard'daki Smart Plug anahtarına dokunulduğunda sırasıyla şunlar olur:

1. **Arayüz** — `NeuSwitch`, `DashboardViewModel.onToggle(SmartPlug, false)` fonksiyonunu çağırır; ekranın kendine ait bir durumu yoktur.
2. **ViewModel** — kullanıcının cihaz kontrol yetkisi olduğunu ve prizin çevrimiçi olduğunu kontrol eder, ardından `DeskRepository.setDeviceOn(...)` çağırır ve prizin otomatik kapanma işini iptal eder.
3. **Repository → Room** — yeni durum SQLite'a yazılır. Room'un `Flow`'u yeni satırı yayar; böylece prizin durumunu gösteren her ekran (Dashboard kartı ve ayrıntı sayfası) kendiliğinden güncellenir.
4. **MQTT** — ViewModel, `office/desk/plug_main {"state":"OFF",…}` mesajını `MqttPublisher`'a gönderir. `RecordingMqttPublisher` decorator'ı mesajı iletir ve bir CLIENT log kaydı yazar; 200 ms sonra aynı decorator cihazın "onay"ını DEVICE kaydı olarak yazar (gerçek bir cihazın göndereceği onayın yerine geçer).
5. **Arayüze dönüş** — konsol son mesajı gösterir, Loglar sekmesi iki kaydı da canlı gösterir ve alt bilgideki güç değeri düşer.

Roller

Yetenek	Admin	Operator	Guest
Dashboard'u ve canlı kameraları görme	✓	✓	✓
Cihaz listesini ve cihaz ayrıntılarını görme	✓	✓	✓
Anahtarları, sahneleri ve kaydırıcıları kullanma	✓	✓	–
Kayıtlar	✓	✓	–
Cihazları yeniden adlandırma / taşıma / yeniden başlatma	✓	✓	–
Cihaz ekleme / silme	✓	–	–
Loglar	✓	✓	– (sekme yok)
Kullanıcı yönetimi	✓	–	–

Tablo, `domain/users/User.kt` içindeki `Role.permissions`'tır; ekranlar kullanılamayan şeyleri gizler, ViewModel'ler de işlem yapmadan önce yeniden kontrol eder.

Gerçek bir MQTT broker'ı nereye bağlanır?

Tüm trafik `MqttPublisher` arayüzünden geçer (`data/mqtt/MqttPublisher.kt`). Bugün `LoggingMqttPublisher` mesajı yalnızca log'a yazar ve konsol için son mesajı hatırlar (`TODO(MQTT)` ile işaretli). Gerçek bir istemci (örneğin HiveMQ) aynı arayüzü gerçekleştirir ve `di/AppModule.kt` içinde bağlanır. Broker'dan gelen cihaz olayları `simulation/` klasörünün ve `RecordingMqttPublisher`'ın her komuttan 200 ms sonra yazdığı uydurma "onay"ın yerini alır. Ekranlar ve ViewModel'ler olduğu gibi kalır.

Bilinen bir Android sınırı

"Ekran kilitlenince etkinleştir" (Privacy Guard) seçeneği uygulama çalışırken çalışır. Uygulama arka planda beklerken Android ekran kilidi yayınını bekletir ve uygulamaya dönüldüğünde iletir; kural o zaman uygulanır. Android bu arada uygulamayı kapatmışsa o kilit kaçırılır. Bunu her durumda yakalamak, sürekli görünen bir ön plan servisi bildirimi gerektirirdi; bu demo bilinçli olarak bundan kaçınır. Uygulama içindeki Kullanım Kılavuzu da aynı şeyi söyler.

6. WebView'dan Compose'a

Orijinal widget, JavaFX WebView içinde çalışan HTML/CSS/JS idi. İlk Android sürümü aynı sayfayı bir WebView içinde gösteriyordu. Bu yeniden yazım, onu native Compose ile değiştirir:

- CSS değişkenleri `ui/theme` token'larına dönüştü (aynı renkler, gömülü Roboto ve JetBrains Mono); CSS box-shadow'ları da `Modifier.dropShadow` / `innerShadow` neumorphic modifier'larına dönüştü.
- Bootstrap Icons font üzerinden kullanılır; böylece simgeler orijinaliyle birebir aynıdır.
- `app.js`'teki durum ve sahne mantığı, testleri olan saf bir Kotlin fonksiyonuna (`applyScene`) taşındı.
- Eski web arayüzü tasarım kaynağı olarak `reference/web-ui/` altında tutulur (derlemeye dahil değildir).

7. Testler

`./gradlew test` 323 test çalıştırır (debug birim test görevi, `testDebugUnitTest`): alan kuralları (roller, sahneler, güç, zamanlamalar, gün batımı, CSV, arama), Room repository'leri ve veritabanı migration'ları, sahte repository'ler ve sanal zamanla ViewModel'ler, Robolectric'te Compose ekranları (Hilt ile tüm uygulamanın navigasyonu dahil).

8. Kütüphaneler ve lisanslar

Kütüphane / varlık	Lisans
Jetpack Compose (BOM 2026.09.00), Material 3, Navigation, Lifecycle	Apache 2.0
Hilt 2.60.1, Room 2.8.5, DataStore 1.3.0, WorkManager 2.12.0	Apache 2.0
Media3 ExoPlayer 1.11.1, Glance 1.2.0, kotlinx.serialization, kotlinx.coroutines	Apache 2.0
Roboto, JetBrains Mono (`res/font`)	SIL OFL 1.1 — `licenses/`
Bootstrap Icons (font)	MIT — `licenses/`
Demo kamera klipleri (`res/raw`)	`tools/make_clips.sh` ile FFmpeg'in sentetik kaynaklarından üretildi; üçüncü taraf görüntü yok — `licenses/VIDEO-CLIPS.txt`
Bu uygulama	MIT — `LICENSE`
Orijinal widget	`LICENSE-original-widget`

9. Sözlük

Terim	Anlamı
Composable	Arayüzün bir parçasını tanımlayan fonksiyon; okuduğu durum değiştiğinde Compose onu yeniden çağırır (recomposition).
State hoisting	Durumu çağıran tarafta (genelde ViewModel'de) tutup değerleri ve callback'leri aşağı aktarmak; böylece bir composable kolayca önizlenir ve test edilir.
ViewModel	Ekran döndürmeden etkilenmez; ekran durumunu tutar ve işleri `viewModelScope` içinde başlatır. Ekran tamamen kapanınca bu kapsam iptal edilir.
Flow / StateFlow	Zaman içinde akan bir değer dizisi; `StateFlow`'un her zaman güncel bir değeri vardır ve arayüz onu `collectAsStateWithLifecycle` ile toplar.
Hilt	Bağımlılık enjeksiyonu: sınıflar ihtiyaç duyduklarını constructor'larında ister, Hilt sağlar (testlerde yerine sahteleri verilir).
Room / DAO	Derleme zamanında kontrol edilen SQL ile SQLite; DAO, sorguları içeren arayüzdür.
DataStore	Küçük bir anahtar-değer dosyası (kimin oturum açtığı, turun görülüp görülmediği).
WorkManager / exact alarm	Uygulama kapansa da çalışan arka plan işleri; kullanıcı izin verdiğinde exact alarm'lar tam zamanında tetiklenir.
`remember` / `rememberSaveable`	Bir değeri recomposition'lar boyunca korur; saveable olanı ekran döndürmeden de etkilenmez.
`LaunchedEffect` / `DisposableEffect`	Bir composable'ın ömrüne bağlı olarak bir coroutine çalıştırır ya da bir kaynağı (örneğin video oynatıcı) kurup temizler.